

Tema 7

Clases y Objetos



Índice

- Introducción
- Clases y objetos
- Modificadores de acceso
- Miembros de una clase
- Constructores y destructores
- `this`
- Métodos de instancia y métodos estáticos
- Interfaces
- Relaciones entre clases: Agregación y composición



- Como hemos visto en temas anteriores, La **programación orientada a objetos (POO)** se basa en la idea de modelar un sistema utilizando **objetos** que interactúan entre sí.
- Estos objetos son representaciones de entidades del mundo real o conceptos abstractos y se agrupan en **clases**, que definen sus características y comportamientos.

1. Introducción



- Este enfoque permite que los programas se estructuren de forma más intuitiva, organizando los datos y las operaciones en torno a los objetos que representan entidades, en lugar de realizar tareas de forma secuencial y aislada.
- En este tema vamos a conocer en profundidad las clases y las partes que las componen.

1. Introducción

- En la POO, una clase representa una **entidad** del mundo real o un concepto abstracto y define el diseño que van a cumplir todos los objetos de esa clase.
- Una **clase** es, pues, una plantilla que define la estructura de los objetos de su tipo, mientras que un **objeto** es una instancia concreta de una clase, que ocupa un espacio en memoria y tiene su propio estado.

2. Clases y objetos

- En temas anteriores hemos visto algún ejemplo de clase, por ejemplo definíamos una clase `Car` que representaba un coche y tenía diferentes **propiedades** (marca, modelo, año).
- Puede haber muchos objetos de tipo `Car` diferentes y cada uno de los coches tendrá diferente marca, modelo y año de fabricación.
- Estas clases también pueden contener **métodos** que definan el comportamiento de los objetos que la instancien.

2. Clases y objetos

- Es habitual que nuestros programas se compongan de diferentes clases que comparten un **dominio**.
- Esto quiere decir que las clases se suelen agrupar según el ámbito del problema en grupos que están relacionados conceptualmente.
- Por ejemplo, si estamos programando una aplicación para un hospital, posiblemente nos aparezcan las clases Paciente, Cama, Doctor, Enfermero, Planta, Medicamento... todas relacionadas con el dominio de un hospital.

2. Clases y objetos

- Para **crear una clase** en C#, trabajamos en la ventana del *Explorador de soluciones*.
- Hacemos botón derecho en el proyecto o carpeta donde queremos colocar nuestra clase y elegimos Agregar> Clase.

2. Clases y objetos



- Se nos abrirá un diálogo para elegir la plantilla. Por defecto estará seleccionada la que nos interesa, Clase de C#.
- Elegimos el nombre y ya tendremos creada nuestra clase personalizada:

2. Clases y objetos



- Se nos crea una clase "vacía", con el esqueleto por defecto:

2. Clases y objetos

```
1  using System;
2  using System.Collections.Generic
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace ConsoleApp1
8  {
9      internal class Car
10     {
11     }
12 }
```

- Los **modificadores de acceso** en C# controlan la **visibilidad** y el **alcance** de una clase y de los miembros que la componen.
- Estos modificadores son fundamentales para la **encapsulación**, uno de los principios clave de la programación orientada a objetos (POO), ya que permiten proteger el estado interno de un objeto y controlar cómo se interactúa con él desde el exterior.

3. Modificadores de acceso

- C# ofrece varios modificadores de acceso que determinan qué parte del código puede ver o modificar un miembro de una clase. Los más importantes son:
 - `public`
 - `private`
 - `protected`
 - `internal`

3. Modificadores de acceso

public

- `public` es el modificador de acceso más permisivo.
- Permite que las clases o miembros puedan ser accedidos **desde cualquier parte** del programa, ya sea desde dentro de la misma clase, desde otras clases o incluso desde otros ensamblados.

3. Modificadores de acceso

private

- `private` es el modificador de acceso más restrictivo.
- Los miembros declarados como `private` solo pueden ser accedidos **dentro de la clase** en la que se definen.
- Se utiliza para ocultar detalles de implementación y proteger el estado interno de un objeto, evitando que otros objetos accedan directamente a sus miembros.

3. Modificadores de acceso

protected

- Los miembros declarados como **protected** solo pueden ser accedidos dentro de la clase en la que se definen y sus **derivadas** (herencia, lo veremos en el siguiente tema).
- Se utiliza cuando una clase base quiere compartir detalles con clases derivadas, pero no quiere exponer esos detalles a clases no relacionadas.

3. Modificadores de acceso

internal

- Los miembros declarados como `internal` son accesibles solo dentro del **mismo ensamblado** (proyecto).
- Un ensamblado es una unidad lógica de compilación en C# (generalmente un archivo `.dll` o `.exe`).
- Se utiliza cuando se desea compartir funcionalidades entre varias clases dentro de una biblioteca, pero sin exponerlas a otras bibliotecas.

3. Modificadores de acceso

Resumen

***Nota:** las clases `private` y `protected` únicamente pueden existir dentro de otra clase, nunca por si solas.

3. Modificadores de acceso

Modificador	Misma clase	Clase derivada	Mismo ensamblado	Cualquier sitio
<code>public</code>	✓	✓	✓	✓
<code>private</code>	✓	✗	✗	✗

protected				
internal				

- En C#, los principales miembros de una clase son los **campos**, las **propiedades** y los **métodos**.
- Cada uno de ellos cumple funciones específicas dentro de la estructura de una clase y juega un rol fundamental en la definición del estado y comportamiento de los objetos.

4. Miembros de una clase

Campos

- Los **campos** son variables que se declaran dentro de una clase o estructura y se utilizan para almacenar datos o el estado de un objeto.
- Un campo representa una **propiedad interna** o característica de un objeto y, normalmente, no es accesible directamente desde fuera de la clase.

4. Miembros de una clase

- Aunque los campos pueden llevar asociado cualquier tipo de modificador, por **buenas prácticas** los declararemos `private`.
- Un campo puede ser un tipo primitivo (`int`, `bool`, `string...`) o cualquier otro tipo (clase) personalizada.

4. Miembros de una clase

Ejemplo

- En una clase que represente un coche, podrían ser campos privados, por ejemplo, la marcha que lleva engranada el motor o el aceite de motor, que podría además ser otra clase y tener sus propios campos y propiedades.

4. Miembros de una clase

```
1  public class Car
2  {
3      private int _currentGear;
4      private Oil _engineOil;
5
6      ...
```

Propiedades

- Una **propiedad** es un miembro de una clase que proporciona un mecanismo flexible para leer o escribir el valor de un campo privado.
- Proporcionan una forma segura y controlada de acceder a los valores de los campos privados, permitiendo la lectura y modificación a través de los métodos **get** (para obtener el valor) y **set** (para establecer el valor).

4. Miembros de una clase

- Las propiedades permiten agregar **lógica** de validación, formateo o procesamiento cuando se accede o modifica un campo.
- Podemos crear propiedades de lectura y escritura (con get y set), de solo lectura (únicamente get, pero no set) o de solo escritura (únicamente set, pero no get).
- Las propiedades de solo escritura son poco frecuentes y se suelen usar para restringir el acceso a datos confidenciales.

4. Miembros de una clase

- Las propiedades permiten agregar **lógica** de validación, formateo o procesamiento cuando se accede o modifica un campo.
- Podemos crear propiedades de lectura y escritura (con get y set), de solo lectura (únicamente get, pero no set) o de solo escritura (únicamente set, pero no get).
- Las propiedades de solo escritura son poco frecuentes y se suelen usar para restringir el acceso a datos confidenciales.

4. Miembros de una clase

- Dependiendo de si necesitamos lógica de validación o no, tenemos 2 tipos de propiedades:
- **Propiedades con campos de respaldo** (*backing field properties*): definen los descriptores get y set para acceder a su campo privado.
- **Propiedades implementadas automáticamente** (acortado como *auto-properties*): simplifican el código, que será creado por el compilador.

4. Miembros de una clase

Propiedades con campos de respaldo

- Utilizan un campo de respaldo privado para establecer (set) y recuperar (get) el valor de la propiedad.
- En el bloque set, utilizamos `value` para obtener el valor que se ha asignado a la propiedad.
- Ambos descriptores de acceso pueden realizar validaciones, cálculos, conversiones... antes de devolver o almacenar los datos.

4. Miembros de una clase

- Su sintaxis básica es la siguiente:

4. Miembros de una clase

```
private tipo_dato _propiedad;  
public tipo_dato Propiedad  
{  
    get { return _propiedad; }  
    set { _propiedad = value; }  
}
```

- Como hemos dicho, podemos añadir lógica tanto al `get` (poco frecuente) como al `set` (más frecuente):

4. Miembros de una clase

```
1  private int _hour;
2
3  public int Hour
4  {
5      get { return _hour; }
6      set
7      {
8          if (value < 0 || value > 24)
9          {
10             // Podemos añadir cualquier tipo de lógica
11             // desde modificar el 'value' hasta lanzar una excepción
12             // o simplemente no hacer nada y no modificar
13             return;
14          }
15
16          _hour = value;
17      }
18 }
```

Propiedades automáticas

- Las propiedades automáticas, o más correctamente **propiedades implementadas automáticamente**, son una simplificación del código cuando no es necesario añadir lógica a los descriptores de acceso.
- En este caso, escribimos la propiedad con un get y un set vacíos y sin explicitar el campo de respaldo privado.

4. Miembros de una clase

- El compilador de C# añade de forma transparente el código completo por nosotros.
- También podemos añadir modificadores de acceso a get o set.

Ejemplo:

- En este caso se simplifica el código, pero no tenemos lógica para controlar o validar los valores.

4. Miembros de una clase

```
1 public int Hour { get;
```

Métodos

- Los **métodos** en C# son bloques de código que definen una serie de acciones o comportamientos que puede realizar un objeto (los vimos en detalle el tema anterior).
- Representan las **funcionalidades** o **acciones** de la clase.

4. Miembros de una clase

- Cada método puede realizar tareas, como modificar campos, devolver un resultado o interactuar con otros objetos.
- Así pues, en el contexto de una clase, los métodos, habitualmente, realizan **acciones** y operaciones **con los datos** y propiedades.
- Estas acciones pueden ser tanto para consultar datos como para modificar el estado del objeto.

4. Miembros de una clase

Ejemplo

- Vamos a crear un ejemplo con algo de complejidad que combine campos, propiedades y métodos para asentar lo que hemos visto.
- En este ejemplo vamos a desarrollar la clase Car añadiendo algunos miembros para definir su comportamiento.

4. Miembros de una clase

Ejemplo

- Vamos a crear un ejemplo con algo de complejidad que combine campos, propiedades y métodos para asentar lo que hemos visto.
- En el vamos a desarrollar la clase Car añadiendo algunos miembros para definir su comportamiento.

4. Miembros de una clase

Código

4. Miembros de una clase

```
1  public class Car
2  {
3      private Engine _engine;
4      private List<Wheel> _wheels;
5      private string _vin;
6
7      public string Brand { get; private set; }
8      public string Model { get; private set; }
9      public int Year { get; private set; }
10     public string NumberPlate { get; private set; }
11     public int Kilometres { get; set; }
12
13     public void Drive(int distance)
14     {
15         _engine.Start();
16         Kilometres += distance;
17         _engine.Stop();
18     }
19
20     private void ChangeOil(Oil newOil)
21     {
```

- Esta clase de ejemplo presenta 3 **campos** privados:
- `_engine`: representaría el motor del coche, de tipo `Engine` (otra clase que habríamos desarrollado y contendría sus propios datos y propiedades).
- `_wheels`: lista de otra clase propia (`List<Wheel>`) que representaría los 4 neumáticos del coche.
- `_vin`: número de bastidor (VIN en inglés) que se establece como privado para que no tengan acceso a él desde el exterior.

4. Miembros de una clase

- Cuenta con diferentes **propiedades**:
- Información del coche: Marca (Brand), modelo (Model), año de fabricación (Year), número de matrícula (NumberPlate) y kilómetros (Kilometres).
- Las 3 primeras propiedades son modificables únicamente por la propia clase (el set es privado). La matrícula y el número de kilómetros son modificables desde el exterior.

4. Miembros de una clase

- Implementa los **métodos**:
- **Drive**: representa la conducción de un número de kilómetros pasados como parámetros. Interactúa con el motor (`_engine.Start()` y `Stop`) y **modifica el estado interno** del objeto (actualiza la propiedad `Kilometres`).
- **ChangeOil**: interactúa con el motor y recibe como parámetro otra clase que habríamos implementado.

4. Miembros de una clase

- `GetInfo`: únicamente ofrece información al exterior, pero no modifica el estado del objeto. Devuelve una cadena que concatena varias de las propiedades del objeto.
- Este método podría ser una sobrescritura del método `ToString` que vimos que tienen todos los objetos. Lo veremos en el próximo tema.

4. Miembros de una clase

- En C#, los **constructores** y **destructores** son elementos clave en la vida de un objeto.
- Son responsables de inicializar y limpiar los recursos asociados con una clase.
- Los **constructores** inicializan el estado de los objetos cuando se crean y permiten sobrecargas para mayor flexibilidad.
- Los **destructores** se utilizan para liberar recursos antes de que un objeto sea recolectado.

5. Constructores y destructores

Constructores

- Un **constructor** es un método especial que se llama automáticamente cuando se crea una nueva instancia de una clase.
- Su función principal es inicializar los datos y el estado del objeto.

5. Constructores y destructores

Características

- Tienen el mismo nombre que la clase.
- No tienen tipo de retorno (ni siquiera `void`).
- Pueden ser sobrecargados, lo que significa que se pueden definir varios constructores en la misma clase con diferentes parámetros.
- Pueden ser utilizados para establecer valores predeterminados de los campos o realizar cualquier otra tarea de configuración inicial.

5. Constructores y destructores

- Los constructores pueden ser **sin parámetros o con parámetros**.
- Es habitual que los constructores reciban parámetros para inicializar las propiedades o campos de las clases.
- Al igual que con el resto de métodos, podemos tener sobrecarga en los constructores con distintos parámetros.

5. Constructores y destructores

- Todas las clases tienen como mínimo 1 constructor.
- Si no se define ningún constructor, C# crea **por defecto** un constructor sin parámetros que inicializa los campos y propiedades al valor por defecto de cada tipo de dato.
- **¡OJO!** Para los tipos de referencia, el constructor por defecto los dejará a `null`.

5. Constructores y destructores

Ejemplo

- Para la clase anterior, creamos un constructor en el que inicializamos las propiedades de marca, modelo y año y otro en el que también pasamos la matrícula como parámetro para inicializarla.

5. Constructores y destructores

```
1 public Car(string brand, string model, int year)
2 {
3     Brand = brand;
4     Model = model;
```

```
5     Year = year;  
6 }  
7  
8 public Car(string brand, string model, int year, stri  
9 {  
10     Brand = brand;  
11     Model = model;  
12     Year = year;  
13     NumberPlate = numberPlate;
```

Destruyores

- Un **destructor** en C# es un método especial que se utiliza para realizar tareas de limpieza cuando un objeto es destruido.
- A diferencia de los constructores, únicamente puede haber un destructor por clase.
- El destructor no admite parámetros ni sobrecarga.

5. Constructores y destructores

- Al contrario que los constructores, los destructores **no se utilizan con frecuencia**, ya que el Garbage Collector (GC) de .NET se encarga de liberar automáticamente la memoria no utilizada.
- Sin embargo, en algunos casos, los destructores son útiles para liberar recursos no administrados, como archivos, conexiones de red o bases de datos.
- Actualmente se prefiere el uso del patrón *Dispose* que el uso de destructores.

5. Constructores y destructores

Características

- Tienen el mismo nombre que la clase, pero precedido por una tilde (~).
- No tienen parámetros ni tipo de retorno.
- No se pueden invocar explícitamente; el destructor es llamado **automáticamente** por el recolector de basura cuando el objeto ya no es necesario.
- Solo puede haber un destructor por clase, no puede sobrecargarse ni heredarse.

5. Constructores y destructores

Ejemplo

- El destructor lleva ~ delante del nombre de la clase y se llama automáticamente por el GC, no se puede invocar.

5. Constructores y destructores

```
1 ~Car()  
2 {  
3     Console.WriteLine("Destructor de coche");  
4     // Aquí liberaríamos recursos, como archivos abiertos o conexion  
5 }
```

Vídeo práctico

5. Constructores y destructores

Vídeo práctico - código VideoGame

5. Constructores y destructores

```
1  public enum System
2  {
3      PlayStation5,
4      XboxSeries,
5      PlayStation4,
6      XboxOne,
7      Switch,
8      PC
9  }
10
11 public class VideoGame
12 {
13     private int _timesRented;
14     private double _totalEarnings;
15
16     public VideoGame(string name, System console)
17     {
18         Name = name;
19         Console = console;
20     }
21
22     ... ..
```

Vídeo práctico - código Renting

5. Constructores y destructores

```
1  public class Renting
2  {
3      public Renting(VideoGame game, string client, int dura
4      {
5          VideoGame = game;
6          Client = client;
7          Duration = duration;
8          Date = DateTime.Now.ToShortDateString();
9      }
10
11      ~Renting()
12      {
13          Console.WriteLine("Destruído objeto Renting");
14      }
15
16      public VideoGame VideoGame { get; set; }
17      public string Date { get; set; }
18      public int Duration { get; set; }
19      public string Client { get; set; }
20 }
```

Vídeo práctico - código Shop

5. Constructores y destructores

```
1  public class Shop
2  {
3      public Shop(string name, string address)
4      {
5          Name = name;
6          Address = address;
7
8          Games = new List<VideoGame>();
9          Rentings = new List<Renting>();
10     }
11
12     public string Name { get; set; }
13     public string Address { get; private set; }
14     public List<VideoGame> Games { get; set; }
15     public List<Renting> Rentings { get; set; }
16
17     public void AddVideogame(VideoGame game)
18     {
19         game.IsAvailable = true;
20         Games.Add(game);
21         Console.WriteLine("Añadido videojuego {0} ({1}) a
```

Vídeo práctico - código Program

5. Constructores y destructores

```
1  internal class Program
2  {
3      static void Main(string[] args)
4      {
5          Shop shop = new Shop("Videojuegos AB", "C/AAA, 24. Aa
6
7          shop.AddVideogame(new VideoGame("Red Dead Redemption :
8
9          VideoGame swOutlaw = new VideoGame("Star Wars Outlaw"
10         VideoGame alien = new VideoGame("Alien Isolation", Sy
11         VideoGame fifa20 = new VideoGame("FIFA 20", System.Pl
12         VideoGame witcher = new VideoGame("The Witcher 3", Sy
13         shop.AddVideogame(swOutlaw);
14         shop.AddVideogame(alien);
15         shop.AddVideogame(witcher);
16         shop.AddVideogame(fifa20);
17
18         shop.StartRenting(alien, "Juan Pérez", 5);
19         shop.StartRenting(swOutlaw, "Ana López", 2);
20
21         shop.EndRenting(alien);
22     }
```

- La palabra clave `this` en C# se refiere a la **instancia actual de la clase** en la que se está utilizando.
 - Se utiliza para eliminar conflictos en el nombrado en varios contextos, principalmente:
 - Diferenciar entre parámetros y miembros de clase.
 - Llamar a otros constructores desde el constructor actual.
 - Pasar la instancia de un objeto como argumento a un método
- ## 6. **this**

Diferenciar entre miembros de la clase y parámetros de un método o constructor

- Cuando un parámetro de un constructor o método tiene el mismo nombre que un campo de la clase, el parámetro "oculta" al miembro de la clase.
- Usando la palabra clave `this`, nos referimos al miembro de la clase en vez del parámetro.

6. `this`

Ejemplo

- En el ejemplo de la izquierda **NO ASIGNAMOS** valor a las propiedades Brand y Model, ya que el parámetro las oculta.
- En el ejemplo de la derecha, usando `this` nos referimos **explícitamente** a los miembros de la clase (propiedades en este caso).

6. `this`

```
1 public Car(string Brand, string Model)
2 {
3     Brand = Brand;
4     Model = Model;
```

```
1 public Car(string Brand, string Model)
2 {
3     this.Brand = Brand;
4     this.Model = Model;
```

```
2 {  
3   this.Brand = Brand;  
4 }
```

- Si utilizamos el código de estilo que vimos en el primer tema, no será necesario el uso de `this` en este caso.
- Los campos se escriben en `_camelCase` con `_` delante, las propiedades en `PascalCase` y los parámetros (y variables locales) `camelCase` sin guion.
- De esta manera, evitamos el conflicto entre nombres de parámetros de un método o constructor y miembros de la clase.

6. this

Llamar a otros constructores desde el constructor actual

- En C# podemos llamar a un constructor desde otro. Esto se conoce como **encadenamiento de constructores**.
- Para ello utilizamos la estructura
: `this(parametros)`, donde el número y tipo de parámetros deben coincidir con los de otro constructor.
- Podemos utilizar los parámetros pasados al constructor actual para pasarlos al que invocamos con `this` o utilizar otros diferentes.

6. **this**

Ejemplo

- En este caso, un constructor con un solo parámetro llama al constructor con 3 parámetros, que asigna en la llamada del `this`.
- Se ejecuta en primer lugar el código del constructor invocado con `this` y en después el código del constructor actual.

6. `this`

```
1 public Car(string model) : this("Marca desconocida",  
2 {  
3     Console.WriteLine("Warning: Propiedad 'Brand' no
```

Pasar una instancia como parámetro

- En ocasiones, debemos pasar un objeto del tipo de la clase como argumento de un método.
- Para pasar la referencia de la instancia actual de la clase, debemos utilizar la palabra clave `this`.

6. **this**

Ejemplo

- Vamos a suponer una clase `VehicleAgency` que representaría el organismo de control de vehículos, como la DGT en España.
- En esta clase habría definido un método `RegisterCar(Car car, string numberPlate)` para matricular un coche.
- La clase `Car` tendría ahora un campo `_vehicleAgency` de este tipo.

6. this

Ejemplo

- En este método, asignamos la matrícula a nuestra propiedad de clase y además llamamos al método `RegisterCar` que hemos visto.
- Para indicar que la instancia que vamos a registrar es la actual, utilizamos la palabra `this`.

6. `this`

```
1 public void RegisterNumberPlate(string num)
2 {
3     NumberPlate = numberPlate;
4     vehicleAgency.RegisterCar(this, num);
}
```

Vídeo práctico

6. this

Vídeo práctico - código Book

6. this

```
1  internal class Book
2  {
3      public Book(string title, string author, short year) :
4      {
5      }
6
7      public Book(string title, string author, short year, double price) :
8      {
9          this.Title = title;
10         this.Author = author;
11         this.Year = year;
12         Price = price;
13     }
14
15     public string Title { get; set; }
16     public string Author { get; set; }
17     public short Year { get; set; }
18     public double Price { get; set; }
19
20     public void SetPrice(double Price)
21     {
22         this.Price = Price;
```

Vídeo práctico - código BookUtils y Program

6. this

```
1 internal class BookUtils
2 {
3     public static void ShowDetails(Book book)
4     {
5         Console.WriteLine("Título: {0}\nAutor: {1}\nAño: {2}", b
6     }
7 }

1 internal class Program
2 {
3     static void Main(string[] args)
4     {
5         Book lotr = new Book("El Señor de los Anillos", "JRR Tol
6         lotr.ShowDetails();
7     }
8 }
```

- Hasta ahora, todos los métodos que hemos visto han sido de instancia*. Sin embargo, en C# podemos clasificar los métodos entre de instancia o estáticos.
- Ambos tipos de métodos tienen características y usos distintos, lo que les permite manejar diferentes escenarios en la programación orientada a objetos.
- *Excepto en la clase Program, donde únicamente podemos utilizar métodos estáticos.

7. Métodos de instancia y métodos estáticos

Métodos de instancia

- Los **métodos de instancia** están asociados a una **instancia de una clase**, lo que significa que estos métodos dependen de un objeto de esa clase para ser llamados.
- Cuando se llama a un método de instancia, tiene acceso a los campos y propiedades de la instancia, y puede manipular el estado del objeto.

7. Métodos de instancia y métodos estáticos

- Los métodos de instancia **requieren** una instancia del objeto: para invocar un método de instancia, primero se debe crear una instancia de la clase a la que pertenece el método.
- Estos métodos tienen **acceso** a otros métodos y campos de instancia de la clase.
- También pueden usar la palabra clave `this` para hacer referencia al objeto actual.

7. Métodos de instancia y métodos estáticos

Métodos estáticos

- Los **métodos estáticos** están asociados a la **clase en sí misma**, en lugar de a una instancia de la clase.
- Esto significa que se invocan sin crear un objeto de esa clase.
- Los métodos estáticos se utilizan cuando la lógica de un método no depende del estado de una instancia particular y puede ejecutarse sin necesidad de crear un objeto.

7. Métodos de instancia y métodos estáticos

- Los métodos estáticos **no requieren** una instancia: un método estático se invoca directamente desde la clase **sin crear un objeto**.
- Los métodos estáticos **no pueden acceder** directamente a campos o métodos de instancia porque no operan sobre una instancia de la clase. Solo pueden acceder a otros métodos o campos estáticos.

7. Métodos de instancia y métodos estáticos

- Se usan para utilidades o funciones que no dependen de datos de instancia, como funciones matemáticas o conversión de datos.
- Para declara un miembro estático, utilizamos la palabra clave `static` delante del tipo de retorno.
- Para invocarlo, utilizamos `Clase.Metodo()` desde cualquier lugar con acceso a la clase y el método.

7. Métodos de instancia y métodos estáticos

Ejemplo

- En esta clase, creamos 2 métodos estáticos para convertir de Km/h a M/s y de millas a kilómetros.

7. Métodos de instancia y métodos estáticos

```
1 public class Conversions
2 {
3     public static double KmHToMs(double kmh)
4     {
5         return kmh * 1000 / 3600;
6     }
7
8     public static double MilesToKilometers(do
9     {
```

```
10 _ return miles * 1.609;
```

- En la clase `Car` añadimos el siguiente método:
- En él, tenemos un parámetro que indica si la distancia viene en millas (por defecto lo hemos puesto a `false`). En caso de que sea `true`, llama al método estático que acabamos de crear para realizar la conversión.

7. Métodos de instancia y métodos estáticos

```
1 public void Drive(int distance, bool isMiles = fal
2 {
3     if (isMiles)
4     {
5         distance = (int)Conversions.MilesToKilomet
6     }
7
8     Drive(distance);
```

- Además de los métodos, también podemos añadir el modificador `static` a campos, propiedades e incluso a la propia clase.
- En el caso de crear una clase estática, únicamente podrá contener miembros estáticos y además no se podrán crear instancias de ella.

7. Métodos de instancia y métodos estáticos

Vídeo práctico

7. Métodos de instancia y métodos estáticos

Vídeo práctico - código ConversionsUtils

7. Métodos de instancia y métodos estáticos

```
1  internal static class ConversionsUtils
2  {
3      public static double MetersToYards(double meters)
4      {
5          return meters * 1.093;
6      }
7
8      public static double MetersToFeet(double meters)
9      {
10         return meters * 3.28;
11     }
12
13     public static double GallonsToLiters(double gallons)
14     {
15         return gallons * 3.785;
16     }
17 }
```

Vídeo práctico - código CollectionObject

7. Métodos de instancia y métodos estáticos

```
1  internal class CollectionObject
2  {
3      public static int _totalObjects;
4
5      public CollectionObject()
6      {
7          _totalObjects++;
8          Console.WriteLine("Objeto creado. Objetos totales: " + _t
9      }
10 }
```

Vídeo práctico - código Program

7. Métodos de instancia y métodos estáticos

```
1  internal class Program
2  {
3      static void Main(string[] args)
4      {
5          Console.WriteLine("5 galones en litros es: " + Conv
6
7          CollectionObject obj1 = new CollectionObject();
8          CollectionObject obj2 = new CollectionObject();
9          CollectionObject obj3 = new CollectionObject();
10         CollectionObject obj4 = new CollectionObject();
11         CollectionObject obj5 = new CollectionObject();
12     }
13 }
```

- Una **interfaz** en C# es un **contrato** que define un conjunto de métodos, propiedades, eventos o indexadores que una clase debe implementar.
- Las interfaces no contienen la implementación del comportamiento, solo la **declaración** de lo que debe hacerse.
- La clase que implemente la interfaz tiene la responsabilidad de definir **cómo** se llevará a cabo ese comportamiento.

8. Interfaces

- Las interfaces son fundamentales en la POO ya que permiten diseñar sistemas modulares y flexibles.
- Proporcionan una manera de garantizar que las clases implementen ciertos comportamientos sin entrar en cómo se implementará ese comportamiento.

8. Interfaces

Características

- **Declaración sin implementación:** En una interfaz solo se declaran métodos, propiedades, eventos, y otros miembros, pero no se define su implementación.
- **Implementación obligatoria:** Cualquier clase o estructura que implemente una interfaz debe proporcionar la implementación de todos los miembros definidos en la interfaz.

8. Interfaces

- **No contienen campos:** Las interfaces no pueden contener campos (variables) o constructores.
- **Acceso público por defecto:** Si no especificamos ningún modificador de acceso, tanto la interfaz como sus miembros son `public`. Además, no se puede utilizar el modificador `private`.

8. Interfaces

Ventajas

- **Polimorfismo:** Las interfaces son una forma poderosa de implementar el **polimorfismo** en C#. Una clase puede implementar múltiples interfaces, lo que permite un comportamiento flexible y modular (lo veremos en detalle en el próximo tema).
- **Flexibilidad:** Diferentes clases pueden implementar la misma interfaz de diferentes maneras, lo que permite escribir código más flexible y reutilizable.

8. Interfaces

- **Desacoplamiento:** Permiten que el código dependa de interfaces y no de implementaciones concretas, facilitando la modificación o extensión del código sin romper la funcionalidad existente.

8. Interfaces

Definición y uso

- La sintaxis básica de la definición de una interfaz es la siguiente:

8. Interfaces

```
modificador_acceso interface INombre  
{  
    // Métodos  
    tipo_retorno Metodo(parametros)  
  
    // Propiedades  
    tipo_dato Propiedad { get; set; }
```

- Para que una clase implemente una interfaz, se utiliza la expresión : `INombreInterfaz` después del nombre de la clase:
- La clase **debe implementar** todos los miembros definidos en la interfaz.
- Si implementa más de una interfaz, se ponen seguidas, separadas por coma (,).

8. Interfaces

```
1 modificador_acceso NombreClase : INo
2 {
3     ...
```

Ejemplo

8. Interfaces

```
1 public interface IPaymentMethod
2 {
3     void ProcessPayment(double quantity)
4 }
5
6 public class CreditCard : IPaymentMethod
7 {
8     public long CardNumber { get; set; }
9     public string Titular { get; set; }
10
11     public void ProcessPayment(double quantity)
12     {
13         // Comunicación con el banco y procesado del pago
14         Console.WriteLine("Pagados {0}€ con la tarjeta {1}", quantity, Titular);
15     }
16 }
```

```
1 public class PayPalAccount : IPaymentMethod
2 {
3     public string Email { get; set; }
4
5     public void ProcessPayment(double quantity)
6     {
7         // Pago a través de la plataforma de PayPal
8         Console.WriteLine("Pagados {0}€ con la cuenta de PayPal {1}")
9     }
}
```

- En el ejemplo anterior hemos definido una interfaz que representa un medio de pago con un único método para procesar el pago.
- Tenemos 2 clases que implementan la interfaz (además de poder tener sus propios miembros aparte), una clase que representa una tarjeta de crédito y otra una cuenta de PayPal.
- Cada una implementa el comportamiento que declara la interfaz atendiendo a sus características.

8. Interfaces

- La ventaja que nos ofrece esta interfaz es que la podremos utilizar independientemente del método de pago.
- Así, podemos tener una clase para gestionar una tienda online, por ejemplo, y simplemente utilizando la interfaz podremos implementar diferentes métodos de pago sin modificar nuestro código.
- Además, estamos abiertos a añadir nuevos métodos en un futuro sin tener que volver a tocar el código de la tienda.

8. Interfaces

- Como vemos, tenemos un carrito de compra que es agnóstico del método de pago que elijamos. Simplemente calculamos el precio y el propio método de pago se encarga de definir su comportamiento.
- Esto nos proporciona una gran flexibilidad y desacoplamiento, características deseables en la POO.

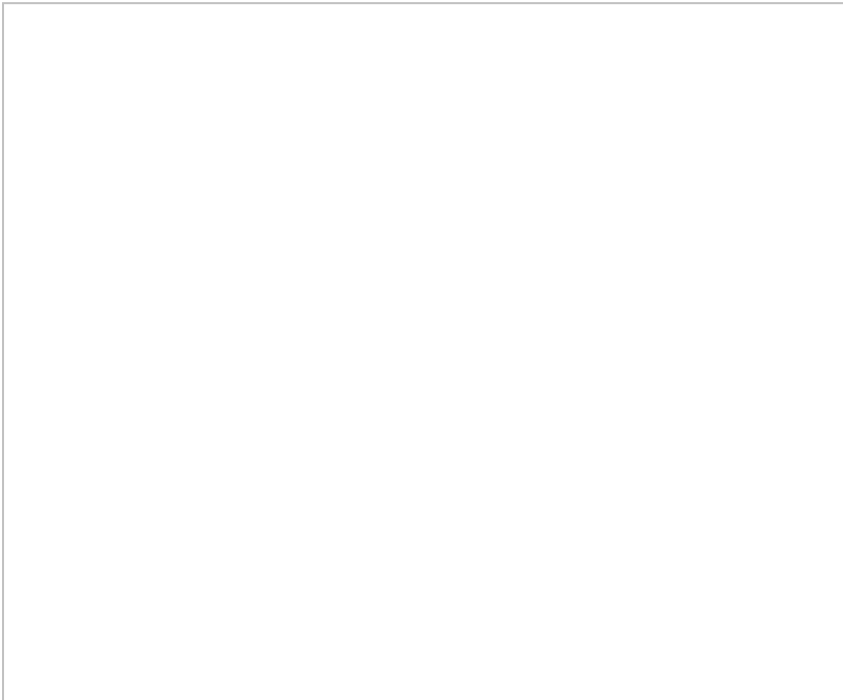
8. Interfaces

```
1 public class ShoppingCart
2 {
3     public IPaymentMethod PaymentMethod { get;
4     public List<Article> Articles { get; set;
5
6     public void Checkout()
7     {
8         double totalPrice = // Suma del precio
9         PaymentMethod.ProcessPayment(totalPrice);
10    }
```

Añadir una interfaz

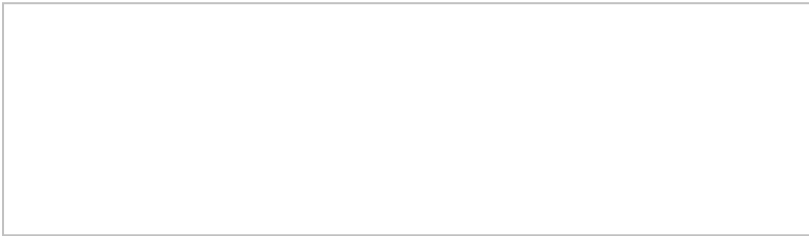
- Para añadir una interfaz, le damos a añadir nuevo elemento desde el explorador de soluciones:

8. Interfaces



- En el nuevo diálogo elegimos mostrar todas las plantillas:
- Y por último seleccionamos 'Interfaz', nombre y *Aceptar*:

8. Interfaces





Vídeo práctico

8. Interfaces

Vídeo práctico - código IProduct

8. Interfaces

```
1 internal interface IProduct
2 {
3     string Brand { get; set;
4     string Name { get; set;
5     double Price { get; set;
6
7     void ShowInfo();
8 }
```

Vídeo práctico - código CoffeMaker

8. Interfaces

```
1  internal class CoffeMaker : IProduct
2  {
3      public CoffeMaker(string brand, string name, double price, int pressure)
4      {
5          Brand = brand;
6          Name = name;
7          Price = price;
8          Pressure = pressure;
9      }
10
11     public string Brand { get; set; }
12     public string Name { get; set; }
13     public double Price { get; set; }
14     public double Pressure { get; set; }
15
16     public void ShowInfo()
17     {
18         Console.WriteLine("Cafetera marca {0}. Modelo {1}\nP");
19     }
20 }
```

Vídeo práctico - código Refrigerator

8. Interfaces

```
1  internal class Refrigerator : IProduct
2  {
3      public Refrigerator(string brand, string name, double pr
4      {
5          Brand = brand;
6          Name = name;
7          Price = price;
8          Capacity = capacity;
9      }
10
11     public string Brand { get; set; }
12     public string Name { get; set; }
13     public double Price { get; set; }
14     public double Capacity { get; set; }
15
16     public void ShowInfo()
17     {
18         Console.WriteLine("Frigorífico {0} de la marca {1}.
19     }
20 }
```

Vídeo práctico - código Main

8. Interfaces

```
1  static void Main(string[] args)
2  {
3      List<IProduct> products = new List<IProduct>();
4
5      Refrigerator refrigerator = new Refrigerator("LG", "Comb
6      CoffeMaker coffeMaker = new CoffeMaker("Saeco", "Barista
7
8      products.Add(refrigerator);
9      products.Add(coffeMaker);
10
11     foreach (IProduct product in products)
12     {
13         product.ShowInfo();
14         Console.WriteLine();
15     }
16 }
```

- En la programación orientada a objetos (OOP), las **relaciones entre clases** son fundamentales para organizar y estructurar el código.
- Estas relaciones permiten definir cómo los objetos interactúan entre sí.
- A lo largo de este tema hemos visto algún ejemplo de clases que contenían campos o propiedades cuyo tipo eran otras clases personalizadas.

8. Relaciones entre clases: Agregación y composición

- En esos casos, decimos que esas clases tienen una **relación entre clases**.
- Dos de las relaciones más comunes son la **agregación** y la **composición**. Ambas representan formas de **asociación** entre objetos, pero difieren en la naturaleza y el ciclo de vida de los objetos involucrados.

8. Relaciones entre clases: Agregación y composición

Agregación

- La **agregación** es una relación en la que una clase contiene o posee objetos de otras clases, pero esos objetos pueden existir de manera independiente.
- En otras palabras, los objetos que forman parte de la relación pueden seguir existiendo sin la clase contenedora.
- Esta relación se conoce como una relación "**tiene un**".

8. Relaciones entre clases: Agregación y composición

- El objeto "contenedor" tiene una referencia a uno o más objetos "contenidos", pero esos objetos pueden existir fuera del ciclo de vida del contenedor.
- Es una **relación débil** entre las clases.
- Un ejemplo común sería una **universidad** y los **estudiantes**: una universidad tiene estudiantes, pero los estudiantes pueden existir independientemente de la universidad.

8. Relaciones entre clases: Agregación y composición

Ejemplo

8. Relaciones entre clases: Agregación y composición

```
1  public class Professor
2  {
3      public string Name { get; set; }
4
5      public Professor(string name)
6      {
7          Name = name;
8      }
9  }
10
11 public class Course
12 {
13     public string Name { get; set; }
14     public Profesor AssignedProfessor { get; set; }
15
16     public Course(string name, Professor professor)
17     {
18         Name = name;
19         AssignedProfessor = professor;
20     }
21
22     public void PrintDetails()
```

- En el ejemplo anterior, el objeto Professor es creado de forma independiente y luego asignado al Course. El profesor puede seguir existiendo incluso si el curso es eliminado.
- Es una relación de asociación y se dice que el curso **tiene un** profesor.

8. Relaciones entre clases: Agregación y composición

Composición

- La **composición** es un tipo más fuerte de relación, en la cual una clase está compuesta de objetos de otras clases, y esos objetos no pueden existir de manera independiente fuera de la clase que los contiene.
- Es decir, los objetos contenidos **no tienen sentido fuera del ciclo de vida** de su contenedor.
- Esta relación también se conoce como una relación "**es parte de**".

8. Relaciones entre clases: Agregación y composición

- Los objetos dependientes **no pueden existir** sin el objeto contenedor.
- Es una **relación fuerte** entre las clases.
- Si el objeto "contenedor" es destruido, todos los objetos "contenidos" también son destruidos.
- Un ejemplo común sería una **casa** y sus **habitaciones**: las habitaciones no existen fuera del contexto de la casa; si la casa desaparece, las habitaciones también lo hacen.

8. Relaciones entre clases: Agregación y composición

- Los objetos dependientes **no pueden existir** (o no tiene sentido que existan) sin el objeto contenedor.
- Es una **relación fuerte** entre las clases.
- Si el objeto "contenedor" es destruido, todos los objetos "contenidos" también son destruidos.
- Un ejemplo común sería una **casa** y sus **habitaciones**: las habitaciones no existen fuera del contexto de la casa; si la casa desaparece, las habitaciones también lo hacen.

8. Relaciones entre clases: Agregación y composición

Ejemplo

- En la clase Car que hemos visto, definíamos un motor mediante una clase Engine que no habíamos implementado.
- Este motor únicamente tenía sentido dentro de un coche y por tanto, su ciclo de vida se gestionaría íntegramente dentro de un objeto de la clase coche.
- Así, podríamos crear el objeto Engine en los constructores de Car y no utilizarlo fuera del objeto.

8. Relaciones entre clases: Agregación y composición

8. Relaciones entre clases: Agregación y composición

Ejemplo

- El ciclo de vida de la clase Engine únicamente tiene sentido dentro de un objeto de la clase Car, no tiene sentido por si solo.
- En este caso, decimos que el motor **es parte de** un coche.

8. Relaciones entre clases: Agregación y composición

Modelado conceptual

- Estas restricciones en las relaciones (por ejemplo, *no pueden existir fuera de*) son únicamente a **nivel lógico**.
- Se trata de cómo organizamos el código a nivel estructural.
- Técnicamente, podemos crear cualquier tipo de objeto al que tengamos acceso en cualquier punto, pero imponemos restricciones lógicas para **dar sentido** a nuestras clases.

8. Relaciones entre clases: Agregación y composición

En este tema hemos conocido por fin los conceptos fundamentales en el que se basa la POO: las clases y los objetos.

Speaker notes